

This course has already ended.

« Chapter 16.1: Files and chess ([https://plus.cs.aalto...](https://plus.cs.aalto.fi/studio_2/k2022/)) 16.3 File IO: Reading JSON file » ([https://plus.cs.aalt...](https://plus.cs.aalto.fi/studio_2/k2022/w16/))
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 16 (https://plus.cs.aalto.fi/studio_2/k2022/w16/)
/ Chapter 16.2: More about text file formats

Chapter 16.2: More about text file formats

About this page:

Key questions: How to create structure for text files?

Topics: Defining human readable formats for text files.

What will I do? Read about text formats and solve a larger programming task.

Indicative difficulty level: Moderate

Rough estimate of workload? 4-5 hours

Point value: S180

Related projects: Chess 
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/).

Task: Reading a human-writeable file (180p)

Assignments in this and the previous chapter include reading data stored in different formats in a file. In both chapters, readable data describes chess game positions. Classes describing the chess game and draft classes for reading a file are given.

Learning goals

- Practice reading from a data stream.
- Learn how to store information in a file.
- A central part of the bigger program is written. In nearly all projects, the program should be able to read from and write into files. In many projects, the format may be simpler.

- Learn about tools in `String` class. In this assignment, tokens have to be searched from strings, strings must be split into parts, extra whitespace (spaces, tabulators, line breaks, etc.) must be removed, and so on.

Intro: File Formats

As we discussed in the previous section, one key feature of nearly all programs is the ability to save information about the program's activities into files, as well as read data from files into the program. Saved data almost always has a clear structure, called a `file format`.

The previous section focused on machine-readable formats. Here we focus on formats that are better suited for human reading and writing.

Manually written format

In this assignment, you will write a program that reads the manually editable file format. For example, many configuration files are like this. They are easy to read, write and modify manually. These formats are also easily extendible: we can add such information which the program does not necessarily need to understand (such as settings for plugins of the program). However, the program should be able to skip such information.

A manual format cannot be as binding as machine-readable formats, and the code reading it should especially tolerate extra whitespace, variation in the order of rows, etc. If a user violates the format rules, it should print as clear a description of the problem as possible, so that he or she can correct the error. (In this exercise, it is enough to throw exceptions; they may be able to carry the information needed to fix the problem.)

Often, we want to be able to include comments in these kinds of files (for example, in Scala code). These should be skipped by the program reading the file. In this assignment, comments are not used.

Task: the format specification

Likewise in the assignment of the previous section, the format of this task has been developed to record chess game positions. Again, we only store the final positions, not moves. An example of a game record is given below followed by a more formal presentation of the format used in the assignment.

This format is designed by following these principles: **easy human readability**, the possibility to add new blocks while still being able to read information with old programs, and fault tolerance (only one piece of information per line). The file also includes a version number and an identifier to prevent situations where the code tries to read the wrong type of data.

Example

The file is intended to be readable and writable by humans. Look at the example below.

```
CHES 1.3 Save file

#Game metadata

Date : 5.7.2001
Black : Marko
White : Lauri

#Comments

Lauri's revenge, not doing great this time either...

#Black

King : a4
Rook : a6
Pawn : b3
Queen : c8

#White

King : d3
Knight : f1
```

chess2clean.sav 

(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/data/chess2clean.sav)

Less polished (but correct) example

A human written file should allow some variation in whitespace usage. The example below showcases a newer version of the file format that can be read with the old version of our program. While reading, we lose information on “febuzzle” and “foobar”, but all features of version 1.3 can still be read normally. In the example, human written text including whitespace is colored yellow. Note the use of empty lines and spaces in the middle and at the end of lines.

```
CHES 1.3 Save file

#Febuzzle
The fnord has been febuzzled.
#game mETAdata

Date:      5.7.2001
Black:#Marko
  White : Lauri

#Comments

Lauri's revenge, not doing great this time either...
Too bad

#Black

Knight    : h4

Pawn      : e3
  Rook:b6
Pawn : c3

#White

King      :      a1

#Foobar
```

chess2.sav  (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/data/chess2.sav)

File Structure

The file structure is the following:

- The Header line contains the following parts separated by individual spaces:
 - "CHES" at the beginning of the line
 - the version number in the form 1.3 (number.number, you can omit the number check in this task),
 - text "Save File"
- This is followed by one or more blocks (starting with the character #, followed by the block identifier).
- Each block ends either at the end of the file or at the next block identifier.
- The character # must be the first character of the line to interpret the line as a block identifier.
- At least the "game metadata", "black" and "white" blocks must be found in order for the game to be downloadable.
- **The "game metadata" block is always before the "black" and "white" blocks.**

Blocks

Structure of the block:

- Tag = character '#' and a string
- Any whitespace at the end of the tag does not belong to the tag.
- Uppercase and lowercase letters in the tag do not matter.
- The block continues until the next '#' character is found, which starts a new block, or until the end of the file/data stream.
- Lines that are empty or include only whitespace are ignored unless the block description tells otherwise. The rest of the lines contain the block data.
- **The lines containing block data may have a variable number of whitespace at both ends of the line or around the separator ' '. **

TIP: You might find it convenient to use the method `trim()` in the `String` class.

Descriptions of the blocks

Comment block (tag `#Comments`)

Skipped in this assignment.

Comment block. The content of the entire block is a string describing the saved position. The empty lines at the beginning and at the end of the text do not belong to the comment. However, empty lines within the text are included.

Game Metadata (tag `#Game Metadata`)

The lines that may be included in this section are described below.

- "Date" ":" D.M.YYYY
 - DATA is the date the game was saved in the format Day.Month.Year
 - **Skipped in this task.**
- "Black" ":" DATA
 - DATA is the name of the black player.
- "White" ":" DATA
 - DATA is the name of the white player.

The above lines may be in any order. The player name may contain spaces, e.g. "Teemu Teekkari" (however, the whitespace at the end of the name does not belong to the name).

Piece Locations (Tags `#Black` and `#White`)

Tag `#Black` starts a block indicating the locations of the black pieces. `#White` tells the locations of white pieces, respectively.

- "King" / "Queen" / "Rook" / "Bishop" / "Knight" / "Pawn" ":" DATA
 - The beginning of the row determines the type of chess piece.
 - DATA is the location of the piece on the board, e.g. "a4".

Chess notation:

The size of the chessboard is 8×8. The board has 8 rows numbered 1–8, and 8 columns marked with the small letters a–h.

A location on the board is represented by concatenating a column and a row marker. For example, "h8" is one of the corners of the board.

Other blocks #???

Unrecognized blocks are likely to have features of the newer format versions. These blocks should be skipped in the task. They should not cause errors.

Error cases

Incorrect piece names, missing players, missing piece location information, etc. are all errors that should lead to throwing a `CorruptedChessFileException`.

Assignment

Implement the method `loadGame` in the class `HumanWritableIO`.

- `def loadGame (input: Reader): Game`

Creates a new `Game` object and returns it. The method reads the game players and the locations of the pieces using the other class methods. At the end of the file, it returns the game object containing the players and the board set in the file. If essential information is missing, the method throws an exception. Comment block and possible unrecognized blocks should be ignored. Create and close the data stream in your test class, not in this method.

If there is something wrong with the file you are reading, the method throws a `CorruptedChessFileException`. The given draft code already throws such an exception in a situation, where there is a problem with the data stream. You should also throw it if there is a problem with the structure of the data.

Given code

Assignment package - Contains the code for the previous and this section's assignments. Chess  (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Chess/). All files except `HumanWritableIO.scala` are same as the ones you used in the assignment in the previous chapter.

- `game.Board.scala` (**Not submitted**)

A very simple class that describes the game board. Allows you to place the pieces on the board and view the locations on the board. Includes an auxiliary method for converting column index letters to grid indexes.

- `game.Player.scala` **(Not submitted)**

A very simple class that describes a player. Allows you to set the name and color of the player and read these attributes.

- `game.Game.scala` **(Not submitted)**

A very simple class that puts together the classes you need in the game. Allows you to place and retrieve players and the game board.

- `game.Piece.scala` **(Not submitted)**

A very simple class that describes a chess piece. A chess piece has an owner (player) and type (King, Knight, etc.)

- `game.io.CorrupctedChessFileException.scala` **(Not submitted)**

An exception class for describing problems in reading a file.

- `game.io.test.BrokenReader.scala` **(Not submitted)**

An auxiliary class for testing error situations.

- `game.io.HumanWritableIO.scala` **(To be submitted)**

An auxiliary class that can be used to load and save (in this exercise saving is not implemented) a game position from a file.

In addition to the methods required for the class, you can also implement auxiliary methods. For example, it may be convenient to read different types of blocks in their own methods.

- `game.io.test.IOTest.scala` **(Not submitted, but this can be very useful.)**

Test file draft.

Submission

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 5 (HumanWritableIO)

This course has been archived (Thursday, 1 June 2023, 12:00).
Select your files for grading

 `HumanWritableIO.scala`

Show anyway

Choose File No file chosen

Submit

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

Show anyway

119

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

nice

Submit an update

